

Data Cleaning Assignment

QUESTION 1

Missing Data Identification

Scenario:

The hospital suspects incomplete patient records.

Task:

Identify missing values in each column

Calculate percentage of missing data

ANSWER

5096	17597	NaN	Female	Chennai	Asthma	2	NaN	0	2023-06-26
5097	19171	NaN	Female	Mumbai	Flu	1	NaN	1	2023-12-31
5098	13854	NaN	Female	Bangalore	Flu	17	NaN	0	2023-01-18
5099	14859	NaN	Male	Delhi	Hypertension	19	NaN	0	2023-08-28

5100 rows x 9 columns

```
[25]: # QUESTION 1
df.isnull().sum()
```

```
[25]: Patient_ID      0
Age             600
Gender          0
City            0
Diagnosis       0
Hospital_Visits 0
Treatment_Cost   593
Insurance_Coverage 0
Admission_Date  0
dtype: int64
```

```
[27]: missing_percentage = (df.isnull().sum() / len(df)) * 100
print(missing_percentage)
```

```
Patient_ID      0.000000
Age             11.764706
Gender          0.000000
City            0.000000
Diagnosis       0.000000
Hospital_Visits 0.000000
Treatment_Cost   11.627451
Insurance_Coverage 0.000000
Admission_Date  0.000000
dtype: float64
```

QUESTION 2

Handling Missing Age

Scenario:

Age is critical for medical analysis, but some values are missing.

Task:

Replace missing Age values with an appropriate method

Justify your choice (mean/median)

ANSWER

```
dtype: int64

[27]: missing_percentage = (df.isnull().sum() / len(df)) * 100
      print(missing_percentage)

Patient_ID      0.000000
Age             11.764706
Gender          0.000000
City            0.000000
Diagnosis       0.000000
Hospital_Visits 0.000000
Treatment_Cost  11.627451
Insurance_Coverage 0.000000
Admission_Date  0.000000
dtype: float64

•[52]: #df.isnull().sum()
      df['Age'] = df['Age'].fillna(median_age)
      print(df['Age'])
      df['Age'].isnull().sum()

0      35.0
1      21.0
2      77.0
3      79.0
4      60.0
...
5095   50.0
5096   50.0
5097   50.0
5098   50.0
5099   50.0
Name: Age, Length: 5100, dtype: float64

[52]: np.int64(0)
```

localhost:8892/notebooks/Untitled20.ipynb?

[53]: df.isnull().sum()

[53]: Patient_ID 0
Age 0
Gender 0
City 0
Diagnosis 0
Hospital_Visits 0
Treatment_Cost 593
Insurance_Coverage 0
Admission_Date 0
dtype: int64

[54]: df

[54]:

	Patient_ID	Age	Gender	City	Diagnosis	Hospital_Visits	Treatment_Cost	Insurance_Coverage	Admission_Date
0	17270	35.0	Male	Bangalore	Hypertension	13	41010.0	1	2023-11-30
1	10860	21.0	Female	Hyderabad	Flu	11	12194.0	1	2023-02-23
2	15390	77.0	Female	Bangalore	Asthma	2	45086.0	0	2023-03-14
3	15191	79.0	Female	Mumbai	Asthma	13	40842.0	0	2023-08-01
4	15734	60.0	Female	Delhi	Asthma	1	9873.0	1	2023-06-20
...	...	...	...	...	...	...	...	...	...
5095	11764	50.0	Female	Mumbai	COVID-19	15	NaN	0	2023-09-22
5096	17597	50.0	Female	Chennai	Asthma	2	NaN	0	2023-06-26
5097	19171	50.0	Female	Mumbai	Flu	1	NaN	1	2023-12-31
5098	13854	50.0	Female	Bangalore	Flu	17	NaN	0	2023-01-18
5099	14859	50.0	Male	Delhi	Hypertension	19	NaN	0	2023-08-28

5100 rows x 9 columns

Filling missing Age values helps improve medical analysis accuracy.

Median imputation prevents distortion caused by extreme age values.

QUESTION 3

Handling Missing Treatment Cost

Scenario:

Treatment cost is highly skewed due to expensive treatments.

Task:

Handle missing Treatment_Cost values

Choose the correct imputation method and explain why

ANSWER

```
localhost:8892/notebooks/Untitled22.ipynb?

[103]: treatment_cost = df['Treatment_Cost'].median()
df['Treatment_Cost'] = df['Treatment_Cost'].fillna(median_cost)
print(df)

   Patient_ID  Age  Gender  City  Diagnosis  Hospital_Visits  \
0         17270  35.0   Male  Bangalore  Hypertension         13
1         10860  21.0  Female  Hyderabad      Flu          11
2         15390  77.0  Female  Bangalore    Asthma           2
3         15191  79.0  Female  Mumbai    Asthma          13
4         15734  60.0  Female   Delhi    Asthma           1
...         ...   ...   ...   ...         ...         ...
5095        11764   NaN  Female   Mumbai  COVID-19          15
5096        17597   NaN  Female  Chennai    Asthma           2
5097        19171   NaN  Female   Mumbai      Flu           1
5098        13854   NaN  Female  Bangalore      Flu          17
5099        14859   NaN   Male    Delhi  Hypertension         19

   Treatment_Cost  Insurance_Coverage  Admission_Date
0         41010.0                   1   2023-11-30
1         12194.0                   1   2023-02-23
2         45086.0                   0   2023-03-14
3         40842.0                   0   2023-08-01
4          9873.0                   1   2023-06-20
...         ...         ...         ...
5095        24797.0                   0   2023-09-22
5096        24797.0                   0   2023-06-26
5097        24797.0                   1   2023-12-31
5098        24797.0                   0   2023-01-18
5099        24797.0                   0   2023-08-28

[5100 rows x 9 columns]

•[105]: #after
df.isnull().sum()

[105]: Patient_ID      0
Age             600
Gender           0
City             0
Diagnosis        0
Hospital_Visits  0
Treatment_Cost   0
Insurance_Coverage  0
Admission_Date   0
dtype: int64
```

QUESTION 4

Duplicate Patient Records

Scenario:

Some patient records were entered multiple times.

Task:

Identify duplicate rows

Remove duplicates

Compare dataset size before and after

ANSWER

BEFORE

[5100 rows x 9 columns]

```
[117]: df.shape
```

```
[117]: (5100, 9)
```

AFTER

The screenshot shows a Jupyter Notebook window with the following content:

```
[135]: print("Duplicate Rows:", df.duplicated().sum())
      #df = df.drop_duplicates()
      df
```

Duplicate Rows: 0

	Patient_ID	Age	Gender	City	Diagnosis	Hospital_Visits	Treatment_Cost	Insurance_Coverage	Admission_Date
0	17270	35.0	Male	Bangalore	Hypertension	13	41010.0	1	2023-11-30
1	10860	21.0	Female	Hyderabad	Flu	11	12194.0	1	2023-02-23
2	15390	77.0	Female	Bangalore	Asthma	2	45086.0	0	2023-03-14
3	15191	79.0	Female	Mumbai	Asthma	13	40842.0	0	2023-08-01
4	15734	60.0	Female	Delhi	Asthma	1	9873.0	1	2023-06-20
...	...	...	...	...	...	...	...	...	...
4996	16135	NaN	Female	Delhi	Hypertension	13	24797.0	1	2023-11-02
4997	15573	35.0	Female	Delhi	COVID-19	19	2585.0	1	2023-12-02
4998	14131	60.0	Female	Chennai	Hypertension	4	31337.0	0	2023-04-07
4999	19900	16.0	Female	Chennai	Flu	16	39801.0	0	2023-08-04
5095	11764	NaN	Female	Mumbai	COVID-19	15	24797.0	0	2023-09-22

5001 rows x 9 columns

```
[136]: df.shape
```

```
[136]: (5001, 9)
```

QUESTION 5

Invalid Age Values (Data Quality Check)

Scenario:

Some patients have unrealistic age values (e.g., >100 or <0).

ANSWER

Task:

Detect such records

Decide whether to remove or correct them

```
[16]: invalid_age = df[(df['Age'] < 0) | (df['Age'] > 100)]
      print(invalid_age)
```

Empty DataFrame

Columns: [Patient_ID, Age, Gender, City, Diagnosis, Hospital_Visits, Treatment_Cost, Insurance_Coverage, Admission_Date]

Index: []

```
[17]: print("Invalid Age Records:", len(invalid_age))
```

Invalid Age Records: 0

QUESTION 6

Outlier Detection (Treatment Cost)

Scenario:

Extreme treatment costs are affecting analysis.

Task:

Detect outliers using IQR method

Display number of outliers

ANSWER

```
jupyter Untitled23 Last Checkpoint: 20 hours ago
File Edit View Run Kernel Settings Help
JupyterLab Python [conda env:base] Anaconda Toolbox

[52]: Q1 = df['Treatment_Cost'].quantile(0.25)
      Q3 = df['Treatment_Cost'].quantile(0.75)

      IQR = Q3 - Q1

      lower_limit = Q1 - 1.5 * IQR
      upper_limit = Q3 + 1.5 * IQR

      outliers = df[
        (df['Treatment_Cost'] < lower_limit) |
        (df['Treatment_Cost'] > upper_limit)
      ]

      print("Number of Outliers:", len(outliers))
      print(outliers)

Number of Outliers: 50
   Patient_ID  Age  Gender  City  Diagnosis  Hospital_Visits  Treatment_Cost  Insurance_Coverage  Admission_Date
53      13152   91.0  Female  Chennai  COVID-19              13      199702.965333              0      2023-01-24
86      10064   20.0  Female  Hyderabad  Hypertension          16      199702.965333              0      2023-01-18
124     17035   59.0   Male  Delhi  Flu              9      199702.965333              0      2023-04-27
183     15855   36.0   Male  Mumbai  Diabetes            1      199702.965333              0      2023-01-16
215     19561   29.0   Male  Mumbai  Flu              16      199702.965333              0      2023-05-31
332     11606   51.0   Male  Delhi  Asthma             16      199702.965333              1      2023-03-31
382     10876   58.0  Female  Bangalore  Hypertension          1      199702.965333              1      2023-10-21
416     14070   NaN    Male  Hyderabad  Flu              6      199702.965333              0      2023-08-18
452     10851   42.0  Female  Bangalore  COVID-19          15      199702.965333              0      2023-06-27
796     18150   37.0  Female  Bangalore  Flu              17      199702.965333              0      2023-05-26
833     19361   41.0  Female  Delhi  Diabetes           19      199702.965333              0      2023-10-08
1127    10943   26.0  Female  Delhi  Asthma            10      199702.965333              0      2023-06-02
1176    18232   29.0   Male  Chennai  Diabetes           10      199702.965333              1      2023-07-10
1219    15408   44.0   Male  Delhi  Hypertension        10      199702.965333              0      2023-04-14
1392    14876   70.0   Male  Mumbai  Diabetes           11      199702.965333              1      2023-12-27
1613    16090   15.0  Female  Hyderabad  COVID-19           2      199702.965333              0      2023-10-07
1729    10575   79.0   Male  Chennai  Diabetes            9      199702.965333              0      2023-11-28
1771    15342   79.0  Female  Bangalore  Diabetes          12      199702.965333              1      2023-08-31
2077    11048   77.0   Male  Chennai  Asthma              8      199702.965333              1      2023-09-30
2166    10593   44.0  Female  Delhi  COVID-19          18      199702.965333              1      2023-02-13
2210    12711   36.0  Female  Mumbai  Diabetes           15      199702.965333              0      2023-03-19
2228    18551   86.0   Male  Bangalore  Hypertension        17      199702.965333              0      2023-11-06
2382    10830   35.0   Male  Delhi  Diabetes           18      199702.965333              1      2023-04-17
2449    10204   0.0    Male  Delhi  COVID-19          12      199702.965333              0      2023-09-20
2531    12113   99.0   Male  Mumbai  Flu              8      199702.965333              1      2023-04-23
2561    13535   NaN    Male  Bangalore  COVID-19          19      199702.965333              0      2023-01-16
2694    14088   56.0  Female  Bangalore  Hypertension          8      199702.965333              0      2023-10-06
2847    17922   NaN    Female  Delhi  COVID-19          12      199702.965333              0      2023-08-01
3213    19120   95.0  Female  Chennai  Flu              2      199702.965333              0      2023-05-12
3279    10674   10.0  Female  Mumbai  Hypertension        17      199702.965333              1      2023-12-09
3361    14781   21.0  Female  Chennai  Diabetes            3      199702.965333              1      2023-10-09
3659    13197   61.0  Female  Chennai  COVID-19          10      199702.965333              1      2023-11-06
3830    18933   15.0  Female  Mumbai  Flu              9      199702.965333              1      2023-09-09
3855    10644   77.0   Male  Delhi  Hypertension        11      199702.965333              1      2023-09-15
3874    19763   86.0  Female  Delhi  Diabetes           16      199702.965333              0      2023-03-30
4869    15944   72.0  Female  Mumbai  Asthma             16      199702.965333              0      2023-02-22
```

QUESTION 7

Outlier Treatment

Scenario:

The business team wants to retain all records.

Task:

Apply capping (Winsorization) on Treatment_Cost

Use 5th and 95th percentile

ANSWER

The screenshot shows a Jupyter Notebook interface with a browser window at the top displaying several tabs: 'youtube - Search', 'Home', 'healthcare_data_cleaning_d', 'Untitled22', 'Home', and 'Untitled23'. The active tab is 'Untitled23', showing a Jupyter Notebook at 'localhost:8915/notebooks/Untitled23.ipynb?'. The notebook contains a data table with columns for patient ID, age, gender, location, disease, treatment cost, and date. Below the table, a code cell [55]: is executed, showing the following Python code:

```
[55]: lower_cap = df['Treatment_Cost'].quantile(0.05)
      upper_cap = df['Treatment_Cost'].quantile(0.95)

      print("5th Percentile:", lower_cap)
      print("95th Percentile:", upper_cap)

      df['Treatment_Cost'] = df['Treatment_Cost'].clip(
          lower=lower_cap,
          upper=upper_cap
      )
      print(df['Treatment_Cost'].describe())
```

The output of the code cell shows the 5th and 95th percentiles of the Treatment_Cost column, followed by a descriptive statistics summary:

```
5th Percentile: 2915.02
95th Percentile: 48187.47
count      4507.000000
mean       25260.711502
std        14351.716597
min        2915.020000
25%        12498.000000
50%        24797.000000
75%        37922.000000
max        48187.470000
Name: Treatment_Cost, dtype: float64
```

The bottom of the notebook shows an empty input cell []:.

QUESTION 9

Time-Based Missing Handling

Scenario:

Admission dates should follow a logical sequence.

Task:

Sort data by Admission_Date

Apply forward fill or backward fill where appropriate

Justify your choice

ANSWER


